

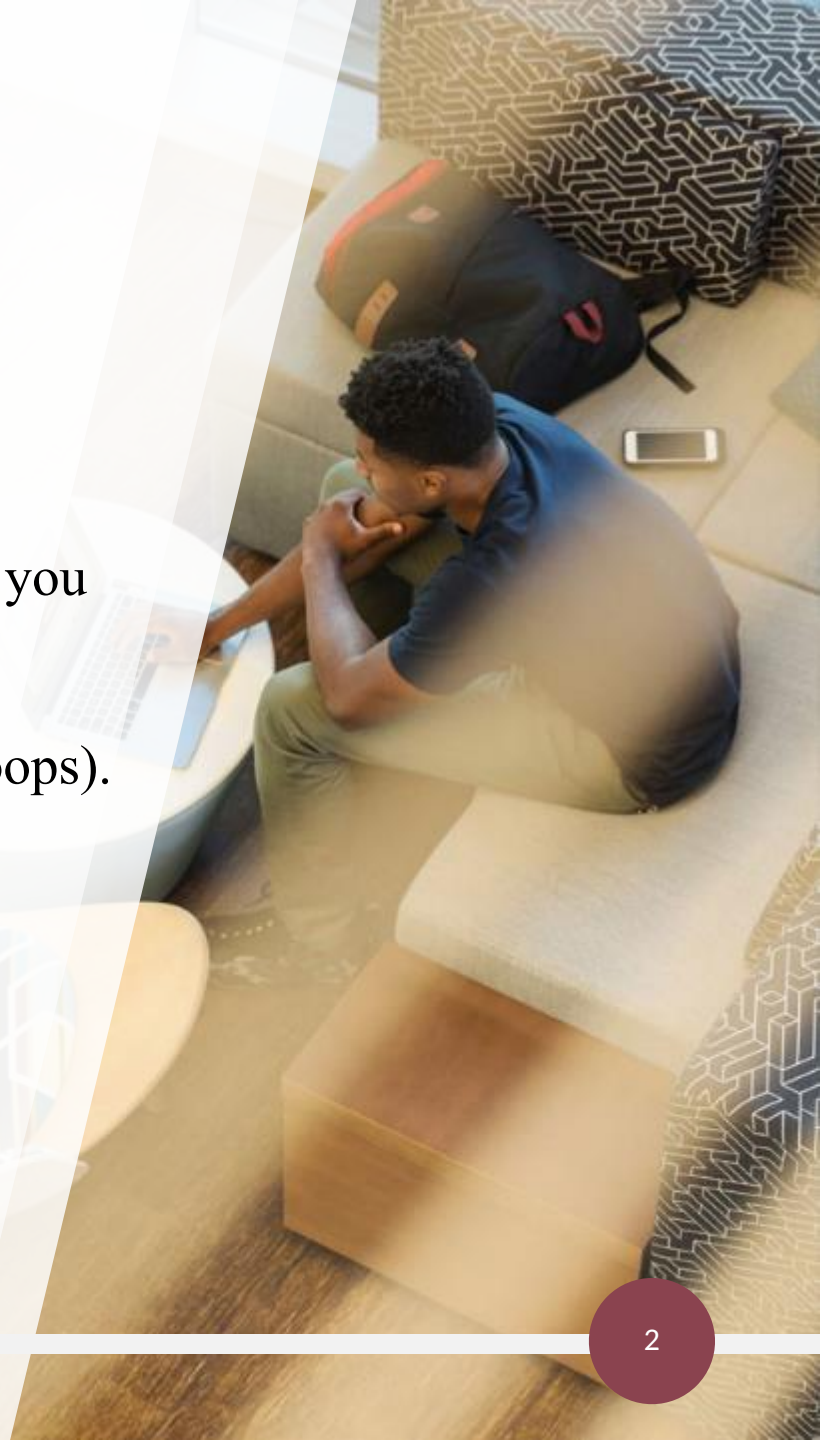
A group of four students are gathered around a table in a library, looking at a laptop screen. The background is filled with bookshelves. A semi-transparent blue diagonal bar and a semi-transparent red horizontal bar are overlaid on the image.

Java 25

Stream Gatherers

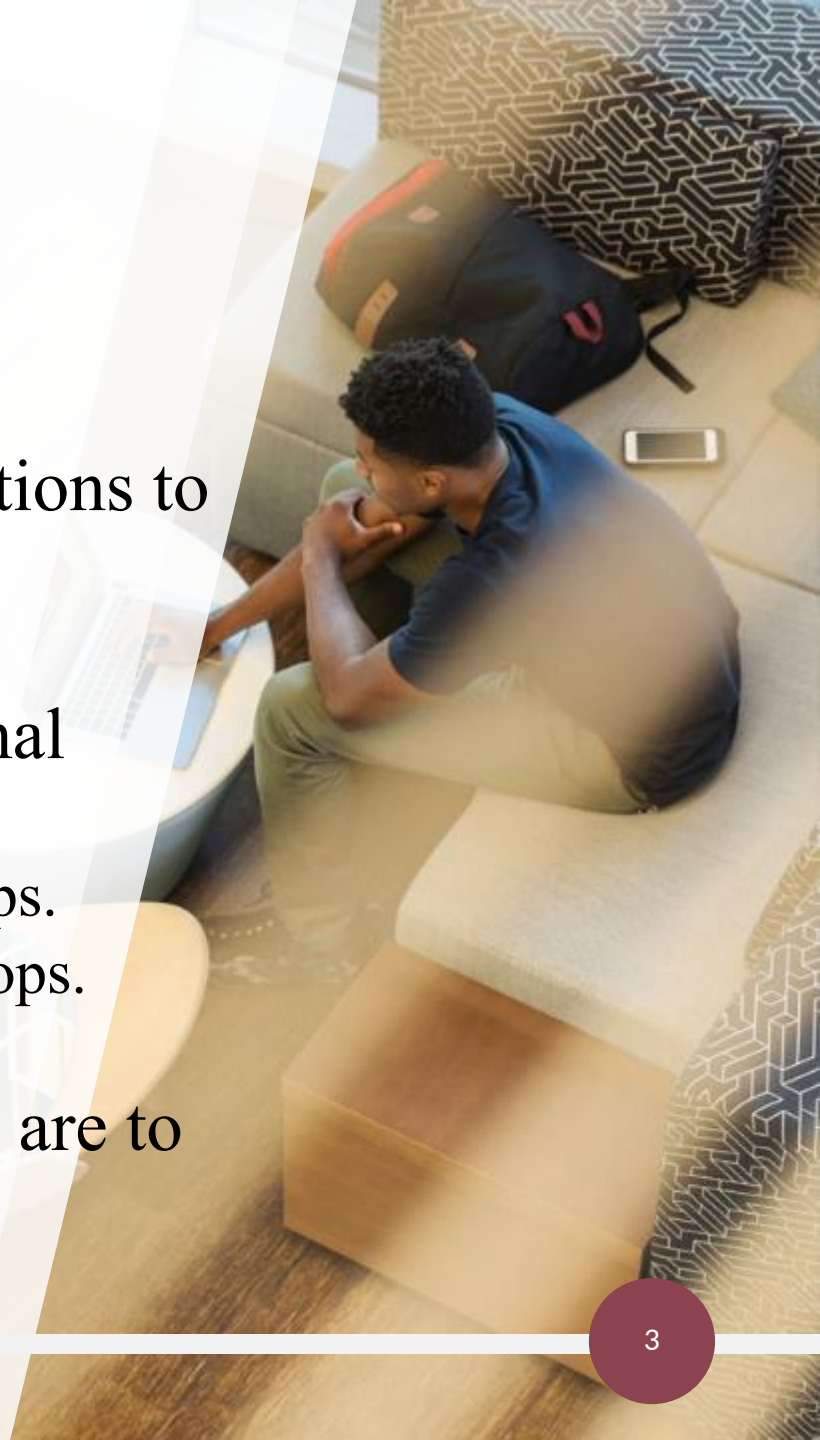
Stream Gatherers - Motivation

- Stream gatherers were finalised in Java 24.
- Prior to stream gatherers, we were restricted to a fixed menu of intermediate operations:
 - This made some “middle-step” patterns awkward:
 - `distinct-by-key` - `distinct()` uses whole object equality; often you want unique by a field (e.g. `User::email`).
 - `batching` - group items into chunks (e.g. 100 at a time).
 - Workaround: collect to a list, then post-process outside the pipeline (loops).
- Now, with stream gatherers, we can define our own custom intermediate operations.
- Enables transformations that don't fit the pre-defined built-in intermediate operations.



Stream Gatherers - Solution

- Over the years, it became clear that the fixed menu of intermediate operations wasn't enough.
- One option would have been to add all these extra operations to the (already large) Stream API.
- A better solution was to use the approach used for terminal operations:
 - `collect(Collector)` is the plug-in point for terminal ops.
 - `gather(Gatherer)` is the plug-in point for intermediate ops.
- Gatherers are to intermediate operations what Collectors are to terminal operations.



Stream Gatherers - Capabilities

- Examine each element as it flows through the stream - enabling “middle-step” logic.
- Track custom state between elements - e.g. unique-by-field, remove consecutive duplicates, running totals.
- Short-circuit if necessary - lets you stop early, including turning infinite streams into finite results.
- Emit 0, 1, or many elements per input.



Stream Gatherers -Anatomy

- A gatherer is a plug-in point for intermediate operations.
 - `stream.gather(myGatherer)` lets us insert our custom gatherer with its custom logic.
- Generic shape: `Gatherer<T, A, R>`
 - T is the input element type (what is flowing in).
 - A = mutable state type (what is tracked as elements flow by).
 - R = output element type (what is flowing out).



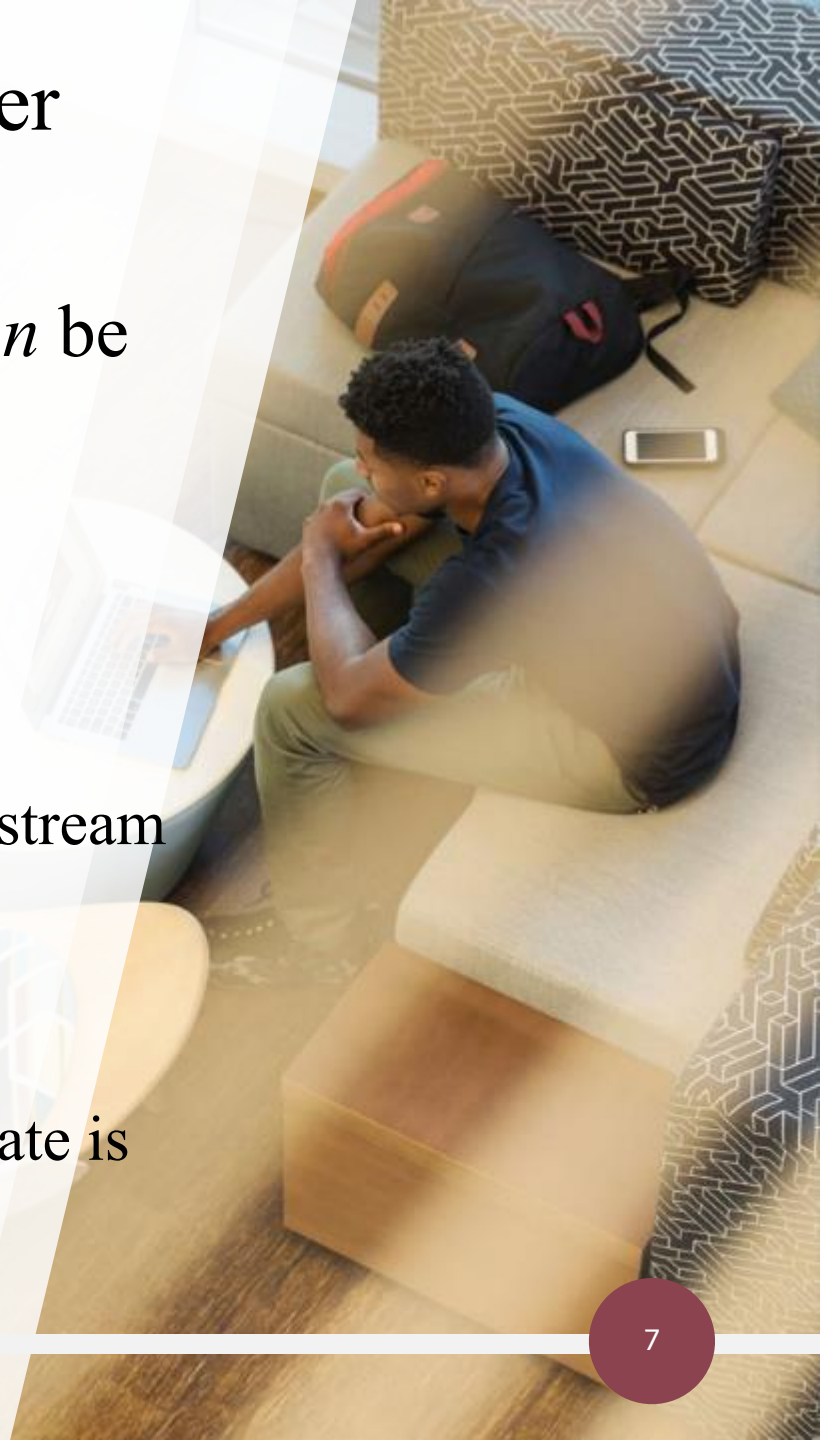
Stream Gatherers - Creating a Custom Gatherer

- `Gatherer.ofSequential(...)`
 - Sequential-only evaluation; use when the stream *can't* be partitioned without changing the result
 - Parameters:
 - `[Initialiser]` - creates the state “memory”
 - omit for stateless; integrator state type is `Void`
 - `Integrator` - custom logic.
 - `[Finisher]` - flush remaining contents if required.



Stream Gatherers - Creating a Custom Gatherer

- `Gatherer.of(...)`
 - Parallel-capable evaluation; use when the stream *can* be partitioned without changing the result
 - Parameters:
 - `[Initialiser]` - creates the state “memory”
 - omit for stateless; integrator state type is `Void`
 - `Integrator` - custom logic.
 - `[Combiner]` - stateful only; merges two states (when the stream is evaluated in parallel).
 - `[Finisher]` - flush remaining contents if required.
- `Stateless.of(...)` overloads have no combiner as the integrator state is `Void` (nothing meaningful to merge).



Stream Gatherers - Creating a Custom Gatherer

- Each gatherer is built from these four functional shapes:
 - Initialiser - creates the state “memory” - *Supplier*
 - `() -> new ArrayList<>() or ArrayList::new`
 - Integrator - per element; updates state; returns boolean - *Integrator*
 - `(state, element, out) -> out.push(element)`
 - Combiner - merges two states when the stream is evaluated in parallel - *BinaryOperator*
 - `(left, right) -> { left.n += right.n; return left; }`
 - Finisher - flushes any final output at the end - *BiConsumer*
 - `(state, out) -> out.push(state.n)`

